

Resumen DSI 1er Parcial

“Etapa de Análisis”

- Repaso POO
- Diagramas UML
- Realización CU de Análisis
- Máquina de Estados
- Patrones GRASP

Repaso POO

Objeto: “Representa un elemento, unidad o entidad individual e identificable, ya sea real o abstracta, con un papel bien definido en el dominio del problema”. Todo objeto tiene un estado, una identidad única y exhibe un comportamiento bien definido (en su clase).

Clase: “Es la definición de un conjunto de objetos que comparten una estructura común y un comportamiento común”. Representa roles, lugares, cosas, conceptos, eventos, transiciones, etc.

Elementos esenciales del Modelo OO:

Abstracción:

Denota características clave de un objeto, dando fronteras conceptuales respecto a la perspectiva del observador.

Encapsulamiento: ¿Qué se oculta?

Proceso de almacenar en un mismo comportamiento, elementos de una abstracción y su implementación.

Modularidad: $AC + BA = Independencia\ de\ componentes$

Propiedad de un sistema descompuesto en un conjunto de módulos cohesivos, débilmente acoplados.

Jerarquía: De Clases (Generalización) – De Partes (Agregación/Composición)

Clasificación u ordenación de abstracciones. (Relaciones de asociación son **NO** jerárquicas).

Relaciones entre clases:

Generalización:

Una clase comparte estructura y/o comportamientos definidos en 1 (herencia simple) o + (herencia múltiple) clases. **Polimorfismo:** Operación de un hijo con misma firma que operación padre, redefiniendo la operación heredada de éste (Varias formas de implementación según clases hijas).

Asociación:

Relación estructural, especifica que los objetos de una clase están conectados con los objetos de otra.

Agregación/Composición (Tipo especial de asociación):

Relación “todo/parte”, donde una clase representa el “todo”, constando de elementos más pequeños (“partes”). En agregación los elementos viven sin su contenedor (Auto-Rueda), en composición no (Cuerpo-Corazón).

Realización

Relación semántica entre clasificadores, en donde un clasificador especifica un contrato que otro clasificador garantiza que cumplirá.

Dependencia

Relación semántica entre dos elementos, en la cual un cambio a un elemento (el elemento independiente) puede afectar a la semántica del otro elemento (elemento dependiente).

Diagramas UML

¿Por qué unificado?

Ciclo de vida de desarrollo: UML proporciona sintaxis visual para modelar software desde los requisitos a la implementación.

Dominios de aplicación: UML se ha utilizado para modelar de todo, desde sistemas incorporados en tiempo real a sistemas de soporte a la toma de decisión.

Lenguajes y plataforma de implementación: UML es neutro tanto en lenguaje como en plataforma.

Procesos de desarrollo: UML puede soportar muchos otros procesos de ingeniería de software además de UP y sus variantes.

Sus propios conceptos internos: UML trata de ser coherente y uniforme en su aplicación de un pequeño grupo de conceptos internos.

UML:

1) *Bloques de Construcción*

a) *Elementos*

Elementos estructurales: son las partes estáticas de un modelo, y representan conceptos o cosas materiales.

1. Clase.
2. Interfaz: colección de operaciones que especifican un servicio de una clase o componente. Define un conjunto de operaciones, pero nunca la implementación de tales.
3. Colaboración: define una interacción y es una sociedad de roles y otros elementos que colaboran para proporcionar un comportamiento mayor que la suma de los comportamientos de sus elementos.
4. Caso de uso: descripción de un conjunto de secuencias de acciones que ejecuta un sistema y que produce un resultado observable de interés para un actor particular.
5. Clase activa: es una clase cuyos objetos tienen uno o más procesos o hilos de ejecución. Clase con comportamiento concurrente.
6. Componente: es una parte modular del diseño del sistema que oculta su implementación tras un conjunto de interfaces externas.
7. Artefacto: es una parte física y reemplazable de un sistema que contiene información física ("bits", ej. código fuente).
8. Nodo: elemento físico que existe en tiempo de ejecución y representa un recurso computacional, que por lo general tiene algo de memoria y capacidad de procesamiento.

Elementos de comportamiento: son las partes dinámicas de los modelos de UML.

1. Interacción: es un comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos, dentro de un contexto particular, para alcanzar un objetivo específico.
2. Máquina de estados: es un comportamiento que especifica las secuencias de estados por las que pasa un objeto o una interacción durante su vida en respuesta a eventos, junto con sus reacciones a estos eventos.
3. Actividad: es un comportamiento que especifica la secuencia de pasos que ejecuta un proceso computacional.

Elementos de agrupación: son las partes organizativas de los modelos UML. Ejemplo:

1. Paquete: es un mecanismo de propósito general para organizar el propio diseño. Es el elemento de agrupación básico para organizar un modelo UML.

Elementos de anotación: son las partes explicativas de los modelos UML. Son comentarios que se pueden aplicar para describir, clasificar y hacer observaciones sobre cualquier elemento de un modelo.

1. Nota: es un símbolo para mostrar restricciones y comentarios junto a un elemento o una colección de elementos.

b) *Relaciones (ya descritas)*

c) *Diagramas*

Diagrama: es la representación gráfica de un conjunto de elementos, visualizado la mayoría de las veces como un grafo conexo de nodos (elementos) y arcos (relaciones). Los diagramas se dibujan para visualizar un sistema desde diferentes perspectivas, de forma que un diagrama es una proyección de un sistema. Un diagrama puede contener cualquier combinación de elementos y relaciones. Sin embargo, en la práctica siempre surgen un número de combinaciones habituales, las cuales son consistentes con las cinco vistas más útiles que comprenden la arquitectura de un sistema. UML se divide en dos tipos de diagramas:

Diagramas Estructurales: Representan elementos y así componen un sistema o función.

1. **Diagrama de clases:** Muestra un conjunto de clases, interfaces y colaboración, así como sus relaciones. Tienen atributos, relaciones y comportamientos. Abarcan la vista de diseño estática de un sistema.
2. **Diagrama de Estructura Compuesta:** Muestra la estructura interna de un clasificador, incluyendo sus puntos de interacción a otras partes del sistema. *Estructura interna:* implementación específica de un componente.
3. **Diagrama de objetos:** Caso especial de DC. Muestra un conjunto de objetos y sus relaciones. Representan relaciones estáticas de instancias de clases. Útiles para la comprensión de DC. Cubren la vista de diseño estática.
4. **Diagrama de componentes:** Representa la encapsulación de una clase, junto con sus interfaces, puertos y estructura interna, formada por otros componentes anidados y conectores. Cubren la vista de implementación estática del diseño de un sistema. Más abstracción que un DC.
5. **Diagrama de despliegue:** Muestra la configuración de nodos de procesamiento (HW) en tiempo de ejecución y los artefactos que residen en ellos. Abordan la vista de despliegue estática de una arquitectura.
6. **Diagrama de paquetes:** Reflejan la organización de paquetes y sus elementos (normalmente DC y DCU).

Diagramas de Comportamiento: Representan características conductuales de un sistema o proceso de negocio.

1. **Diagrama de actividades:** Muestra la estructura de un proceso u otra computación como el flujo de control y datos paso a paso. Cubren la vista dinámica del sistema.
2. **Diagrama de casos de uso:** Captura requisitos de un sistema. Organizan y modelan el comportamiento del sistema. Tienen actores y relaciones. Cubren la vista de casos de uso estática de un sistema.
3. **Diagrama de estados:** muestra una máquina de estados, que consta de estados, transiciones, eventos y actividades. Muestra la vista dinámica de un objeto en respuesta a dichos eventos.
4. **Diagrama de interacción:** muestra una interacción, que consta de un conjunto de objetos o roles, incluyendo los mensajes que pueden ser enviados entre ellos. Cubren la vista dinámica de un sistema.
 - a. **Diagrama de secuencia:** Resalta la ordenación temporal de los mensajes. Muestra objetos como líneas de vida hacia abajo en la página con sus interacciones en el tiempo representadas como flechas desde la línea de vida origen a la línea de vida destino. Buenos para mostrar cuáles objetos se comunican con otros objetos y qué mensajes disparan esas comunicaciones.
 - b. **Diagrama de comunicación:** Resalta la organización estructural de los objetos o roles que envían y reciben mensajes. Secuencia de mensajes representada por esquema numérico.
 - c. **Diagrama de Tiempo:** mostrar los cambios en el estado, o la condición, de una línea de vida de una instancia a lo largo del tiempo, de manera lineal.

d. *Diagrama de Descripción de Interacciones*: diagrama de actividad en el cual los nodos representan diagramas de interacción. Útil para mostrar cómo distintos escenarios se combinan. Pueden incluir más diagramas de interacción dentro.

2) Mecanismos Comunes

Especificaciones, adornos, divisiones comunes, mecanismos de extensibilidad (estereotipos, valores etiquetados, restricciones).

3) Reglas de UML

- Nombres: Cómo llamar a los elementos, relaciones y diagramas.
- Alcance: El contexto que da un significado específico a un nombre.
- Visibilidad: Cómo se pueden ver y utilizar esos nombres por otros.
- Integridad: Cómo se relacionan apropiada y consistentemente unos elementos con otros.
- Ejecución: Qué significa ejecutar o simular un modelo dinámico.

Vistas UML

CU: Describe el comportamiento del sistema a través de los casos de uso. Aspecto estático: Diagrama de casos de uso. Aspecto dinámico: Diagramas de interacción, de estados y de actividades.

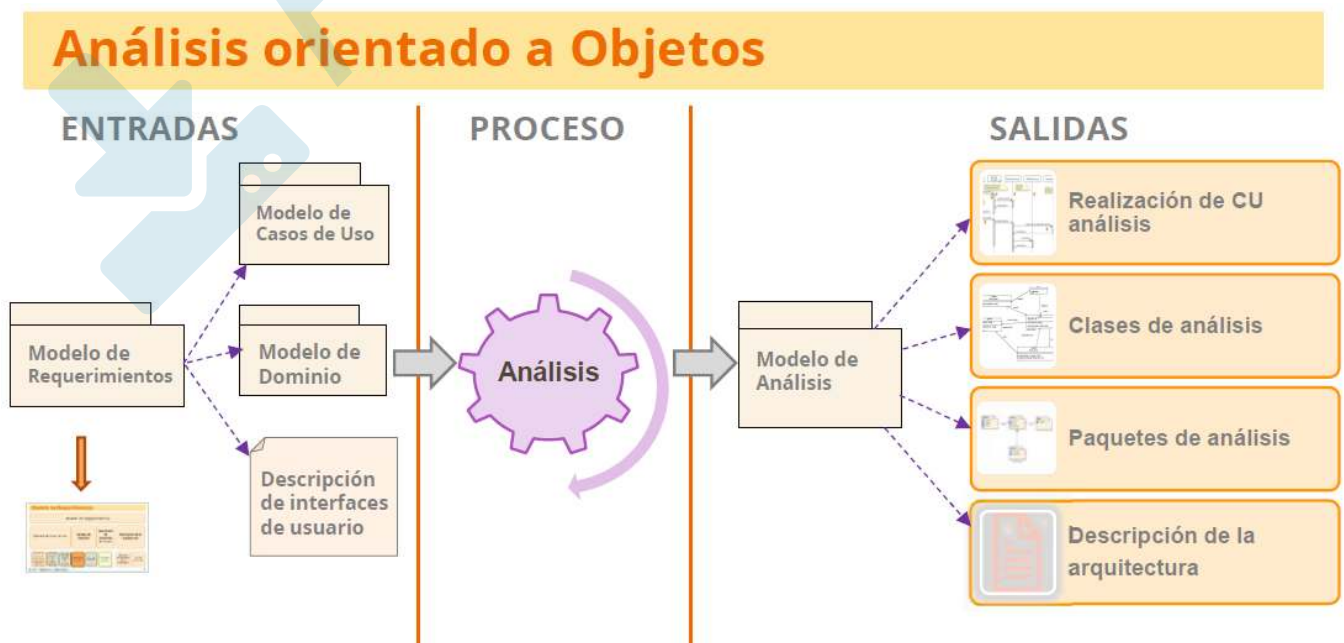
Diseño: Comprende las clases, interfaces y colaboraciones que conforman el vocabulario del problema y la solución. Aspecto estático: Diagramas de clases y objetos. Aspecto dinámico: Diagramas de interacción, de estados y de actividades.

Procesos: Comprende los hilos y procesos que forman los mecanismos de sincronización y concurrencia del sistema. Los aspectos estáticos y dinámicos de esta vista se capturan con los mismos diagramas que la vista de diseño pero poniendo énfasis en las clases activas.

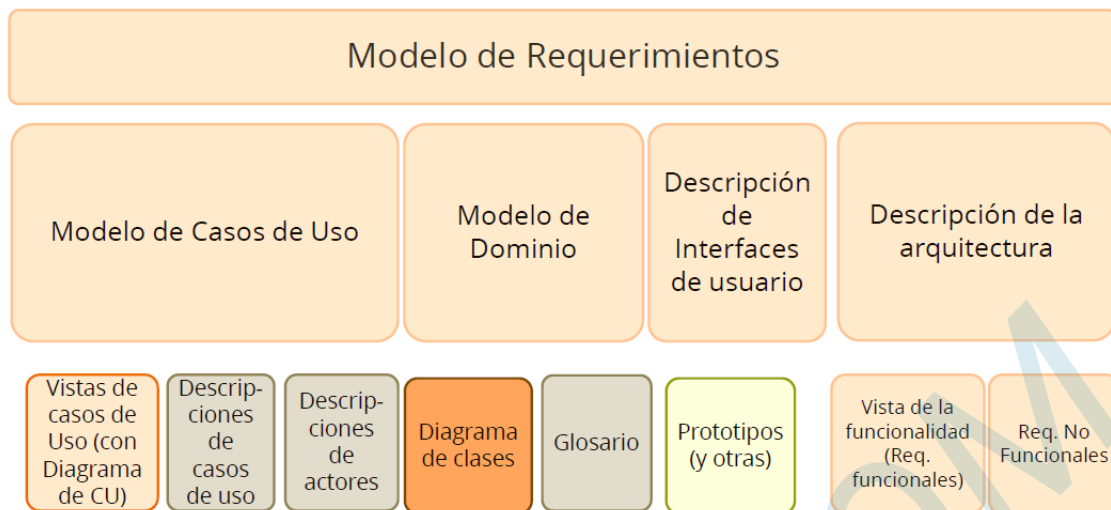
Implementación: Comprende los componentes y archivos que se utilizan para ensamblar y hacer disponible el sistema físico. Aspecto estático: Diagramas de componentes. Aspecto dinámico: Diagramas de interacción, de estados y de actividades.

Despliegue: Contiene los nodos que conforman la topología de hardware sobre la que se ejecuta el sistema. Aspecto estático: Diagramas de despliegue. Aspecto dinámico: Diagramas de interacción, de estados y de actividades.

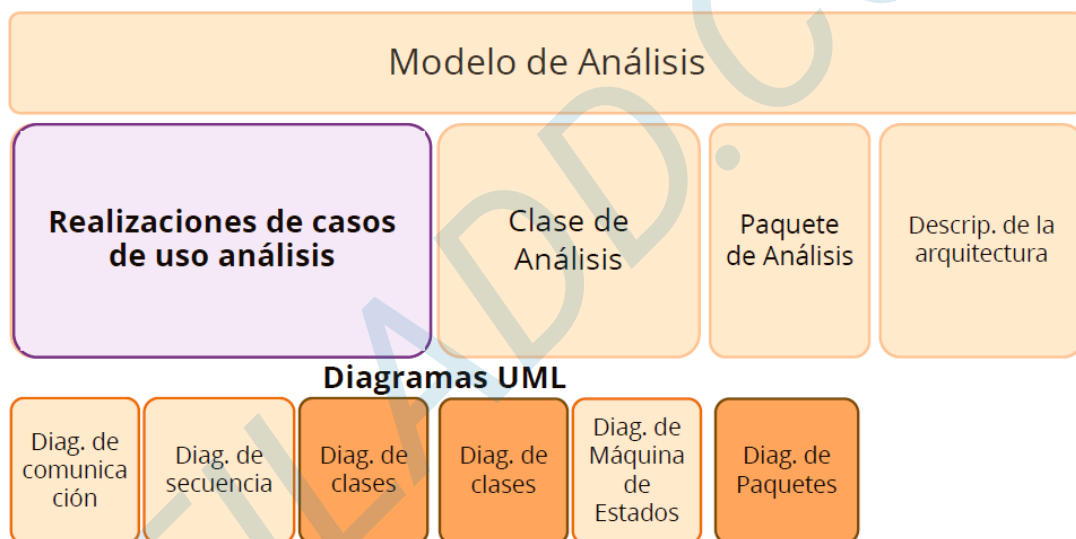
Realización CU de Análisis



Modelo de Requerimientos



Modelo de Análisis



Clases de Análisis:

Representa una abstracción de una o varias clases y/o subsistemas del diseño del sistema. Se centra en el tratamiento de los RF y pospone los RNF. Son evidentes en el contexto del dominio del problema, más “conceptual”.

Su comportamiento se define mediante responsabilidades en un alto nivel y menos formal. No se define mediante una interfaz en términos de operaciones y de sus firmas.

Define atributos en un alto nivel. Los tipos de los atributos son conceptuales y reconocibles en el dominio del problema. En el diseño los tipos pertenecen al lenguaje de programación. Participa en relaciones, aunque esas relaciones son más conceptuales que las de diseño e implementación. Siempre encajan en uno de los tres estereotipos básicos:

Clases de interfaz: se utilizan para modelar la interacción entre el sistema y sus actores. Esta interacción implica a menudo recibir (y presentar) información y peticiones de (y hacia) los usuarios y los sistemas externos. Representan a menudo abstracciones de ventanas, formularios, paneles, interfaces de comunicaciones, interfaces de impresoras, sensores, etc. Deberían asociarse con un al menos un actor y viceversa.

Clases de entidad: se utilizan para modelar información que posee una vida larga y que es a menudo persistente. Modelan la información y el comportamiento asociado a algún fenómeno o concepto, como una persona, un objeto del mundo real, o un suceso del mundo real. En la mayoría de los casos, las clases de entidad se derivan directamente de

una clase de entidad del negocio (o de una clase del dominio). Una diferencia fundamental entre clases de entidad y clases de entidad de negocio es que las primeras consideran objetos manejados por el sistema. En consecuencia, las clases de entidad reflejan la información de un modo que beneficia a los desarrolladores al diseñar e implementar el sistema, incluyendo su soporte de persistencia. Modelan información relevante para el dominio y que dura en el tiempo, siendo candidatas a ser persistentes mediante bases de datos.

Clases de **control**: representan coordinación, secuencia, transacciones, y control de otros objetos y se usan con frecuencia para encapsular el control de un caso de uso en concreto. También se utilizan para representar derivaciones y cálculos complejos, como la lógica de negocio que no puede asociarse con ninguna clase de entidad concreta. Los aspectos dinámicos del sistema se modelan con clases de control, debido a que ellas manejan y coordinan las acciones y los flujos de control principales, y delegan trabajo a otros objetos. Son el nexo entre las clases de entidad y de interfaz, ya que éstas no se relacionan directamente.

Paquete de Análisis:

Proporcionan un medio para organizar los artefactos del modelo de análisis en piezas manejables. Puede estar compuesto de clases de análisis, de realizaciones de casos de uso de análisis y de otros paquetes (recursivamente). Deberían ser cohesivos (contenidos fuertemente relacionados) y deberían ser débilmente acoplados (mínima dependencia de uno de otros).

Tienen las siguientes características:

- Pueden representar una separación de intereses de análisis.
- Deberían crearse basándose en los requerimientos funcionales y en el dominio del problema.
- Probablemente se convertirán en subsistemas en las 2 capas de app. superiores del modelo de diseño, o se distribuirán entre ellos.

Descripción de la Arquitectura:

Contiene una vista de la arquitectura del modelo de análisis, que muestra sus artefactos significativos para la arquitectura.

Artefactos que se consideran SPA:

- La descomposición del modelo de análisis en paquetes de análisis y sus dependencias.
- Las clases fundamentales del análisis (entidad, control e interfaz).
- Realizaciones de caso de uso que describen cierta funcionalidad importante y crítica.

Realización de CU - Diagrama de Secuencia:

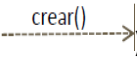
Colaboración que describe cómo se ejecuta un CU, en términos de la interacción entre objetos de análisis.

- Líneas de Vida: Pueden ser de objetos (boundary, control o entity) o instancias de actores (Ej: Encargado).

Sintaxis: **nombre [selector]: Tipo**

- Mensajes: Muestran cómo interactúan los objetos.

Sintaxis (definido en diseño): **nombreMensaje (parámetros: TipoDatoParámetro)**

Tipos: - Síncronos () – Asíncronos () – Creación de Objeto ()
– Señal () – Destrucción de Objeto () – Llamada CU ().

- Notas.
- Fragmentos combinados (loop ("for"), opt ("if"), alt ("else"), etc.). Tienen 1 operador, 1 o + operandos y 0 o + condiciones de protección.

- Foco de Control (Rectángulo largo en línea de vida): Se produce cuando una línea de vida ejecuta un mensaje, indicando así que la línea de vida posee el foco de control.

Tips Importantes para DS

- A nivel análisis, los mensajes se modelan sin parámetros ni valores de retorno.
- La instancia de actor sólo se puede relacionar con el objeto de la clase boundary correspondiente (generalmente una pantalla).
- Mensajes desde un objeto de boundary a un actor: sólo podría suceder en caso que el actor represente un sistema o un dispositivo de hardware.
- Un objeto de entidad no envía mensajes al objeto de control.
- Cuando se trabaja sobre un único objeto, éste se modelará de forma separada a la instancia que representa el conjunto y/o subconjunto de objetos de la misma clase. Ejemplo: “:Pedido” y “seleccionado:Pedido”.
- Quien itera es el objeto de control o un objeto de entidad que debe buscar entre una colección de objetos asociada. Un objeto de una clase NO busca en objetos de la misma clase (a menos que exista una asociación reflexiva).
- Cuando se generan salidas impresas se modelan objetos boundary. Se denominan por lo general ImpresorXX.
- Cuando existe una agregación a nivel de diagrama de clases, no corresponde que el gestor acceda a la parte de dicha agregación, sino que debe el gestor realizar la petición al todo, y éste tiene la responsabilidad de administrar sus detalles. Se aplica así la correcta asignación de responsabilidades y la delegación.

Modelado de Comportamiento con DME

Máquina de Estados: Especificación de la secuencia de estados por las que pasa un objeto a lo largo de su vida en respuesta a eventos junto con sus respuestas a esos eventos. **Toda clase puede tener una sola máquina de estado de comportamiento.**

Elementos Básicos

Estado:

Condición o situación en la vida de un objeto durante la cual satisface alguna condición, realiza alguna actividad o espera algún evento. Representado por: Atributos + Valores Actuales de Esos Atributos. Un objeto permanece en un estado durante una cantidad de tiempo finita. Un estado tiene: Nombre, Efectos de Entrada/Salida, Transiciones Internas, Subestados, Eventos Diferido, Actividades-Do.

ESTADO INICIAL: indica el punto de comienzo por defecto para la máquina de estados o el subestado. Normalmente, se pasa del pseudoestado inicial al estado “real” de la máquina de estado.

ESTADO FINAL: indica que la ejecución de la máquina de estados o del estado que lo contiene ha finalizado.

Transición:

Relación entre dos estados que indica que un objeto que esté en el primer estado realizará ciertas acciones y entrará en el segundo estado cuando ocurra un evento especificado y se satisfagan condiciones especificadas. Una transición tiene: Estado Origen, Evento de Disparo, Condición de Guarda, Efecto, Estado Destino.

Sintaxis: **nombreEvento(lista de parámetros opcional)[condición de guarda opcional]/lista de acciones opcional**

AutoTransición: Estado origen y estado destino son el mismo. En estos casos, un evento dispara la transición, se abandona el estado, se ejecuta una acción (si la hay) y se vuelve a entrar en el mismo estado.

Evento:

Aparición de un estímulo que puede disparar una transición de estado. Puede ser de tipo: Llamada, señal, cambio o tiempo.

Evento de Llamada: Petición de una operación específica a invocarse en una instancia de la clase de contexto. Debería tener la misma firma que una operación en la clase de contexto de la máquina de estado. Recibir un evento de llamada es un activador para que se ejecute la operación.

Estados Compuesto/SubEstado:

Estado Compuesto: Estado con SubEstados. Puede contener subestados concurrentes (ortogonales) o secuenciales (no ortogonales).

SubEstado: Estado anidado dentro de otro.

Permiten simplificar transiciones similares desde diferentes estados, agrupándolos en un estado compuesto.

Heredan **TODAS** las transiciones de los estados en los que están contenidos. Si el estado compuesto tiene una transición determinada, cada uno de los estados anidados dentro de él también tiene esta transición.

SubEstados Secuenciales (No Ortogonales): Dado un conjunto de estados no ortogonales en un estado compuesto sencillo, se dice que el objeto está en el estado compuesto, y sólo en uno de sus subestados (o en el estado final) en un momento dado.

Dado un estado compuesto, una transición desde un origen externo puede apuntar al estado compuesto o a un subestado.

Una transición que salga de un estado compuesto puede tener como origen el propio estado compuesto o un subestado.

Una máquina de estados no ortogonal anidada puede tener como máximo un subestado inicial y un subestado final.

Estado Historia:

Cuando una transición entra en un estado compuesto, la acción de la máquina de estados anidada comienza de nuevo en su estado inicial (a menos que la transición se dirija directamente a un subestado).

Un estado de historia permite que un estado compuesto que contiene subestados secuenciales recuerde el último subestado activo antes de la transición que provocó la salida del estado compuesto. Se representa como un pequeño círculo que contiene el símbolo H.

El pseudoestado de historia superficial puede tener muchas transiciones entrantes, pero solamente una transición saliente. Si una máquina de estados anidada alcanza un estado final, pierde su historia almacenada y se comporta como si aún no se hubiera entrado en ella por primera vez.

Nota: Si queremos conocer todos los estados por los que pasa un objeto de una clase, se resuelve a nivel estructura en el diagrama de clases (Clase CambioEstado)(No afecta al diagrama de máquina de estados).

Trazabilidad de ME con otros Modelos:

En relación al DC

Los métodos (eventos de llamada) que figuran en la ME, y que provocan los cambios de estado, deben estar presentes en la definición de la clase. Debe existir un atributo estado o estar modelado mediante otra clase.

En relación al DCU

Debe existir al menos un CU en relación a cada método que provoca un cambio de estado. Debemos tener CU que nos permitan administrar todos los estados de la clase.

En relación a realizaciones de CU (DS)

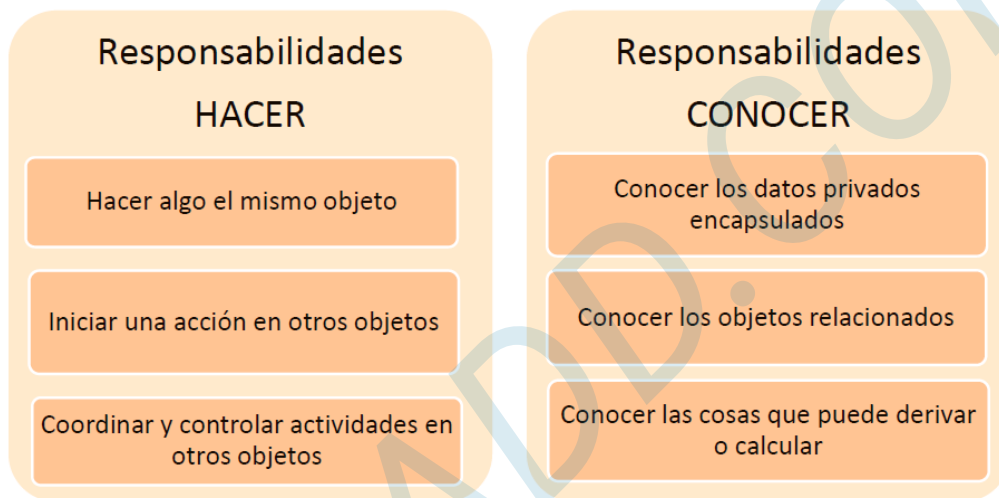
En las realizaciones de CU correspondientes deberán existir mensajes asociados a los métodos que provocan el cambio de estado.

Patrones GRASP

Son patrones para asignación de responsabilidades a los objetos.

La decisión de qué operaciones, dónde colocarlas y cómo deberían interactuar los objetos es muy importante. Se aplican al elaborar los diagramas de interacción.

Responsabilidad se define en UML como “un contrato u obligación de un clasificador”.

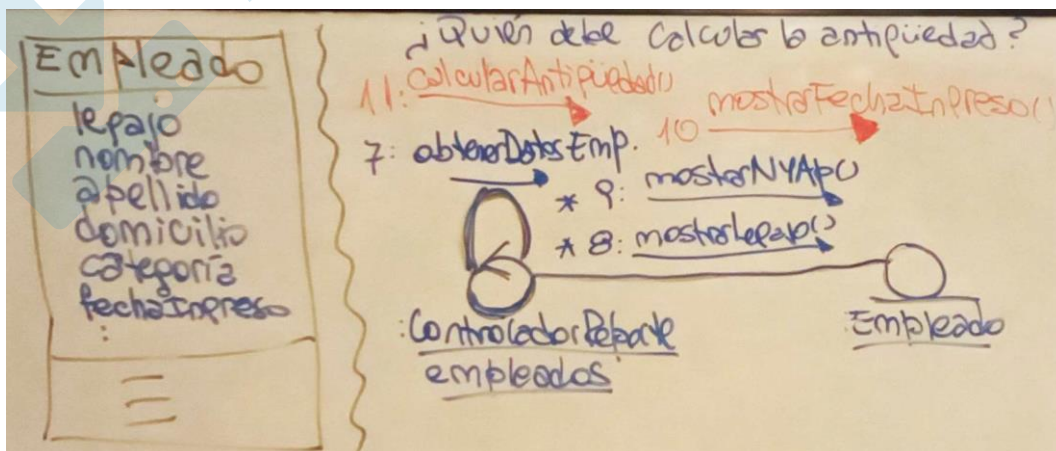


Patrón Experto en Información

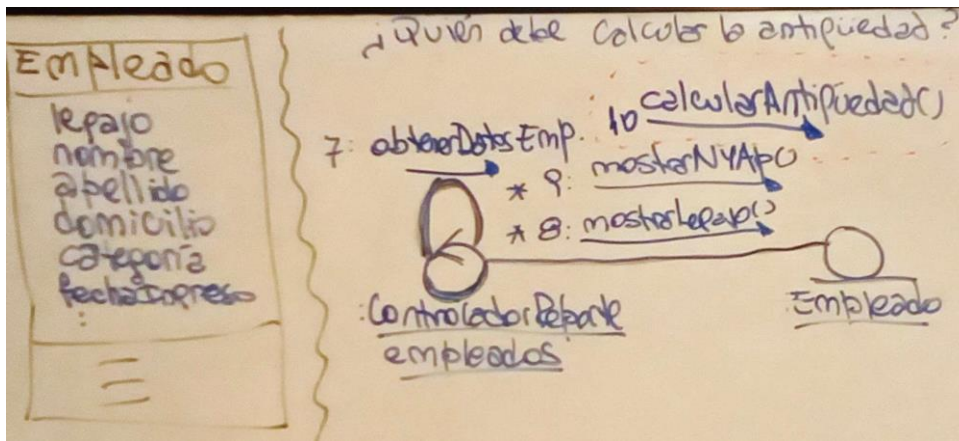
Problema: ¿Cuál es el principio fundamental en virtud del cual se asignan las responsabilidades en el diseño orientado a objetos?

Solución: Asignar una responsabilidad al experto en información: la clase que cuenta con la información necesaria para cumplir con la responsabilidad.

Así NO:



Así Sí:



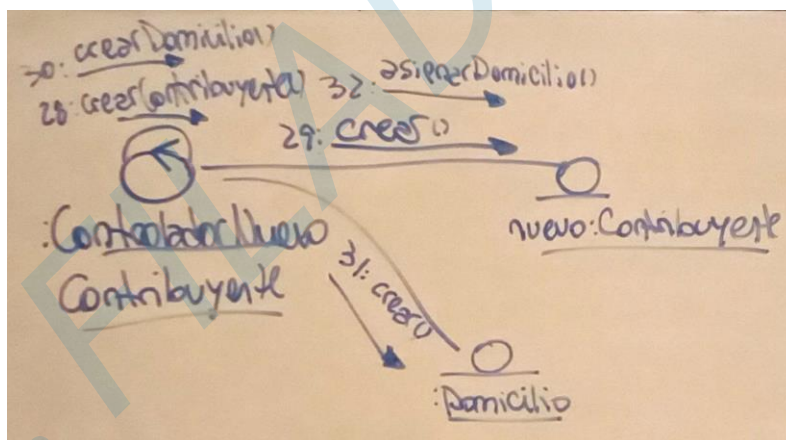
Patrón Creador:

Problema: ¿Quién debería ser el responsable de crear una nueva instancia de alguna clase?

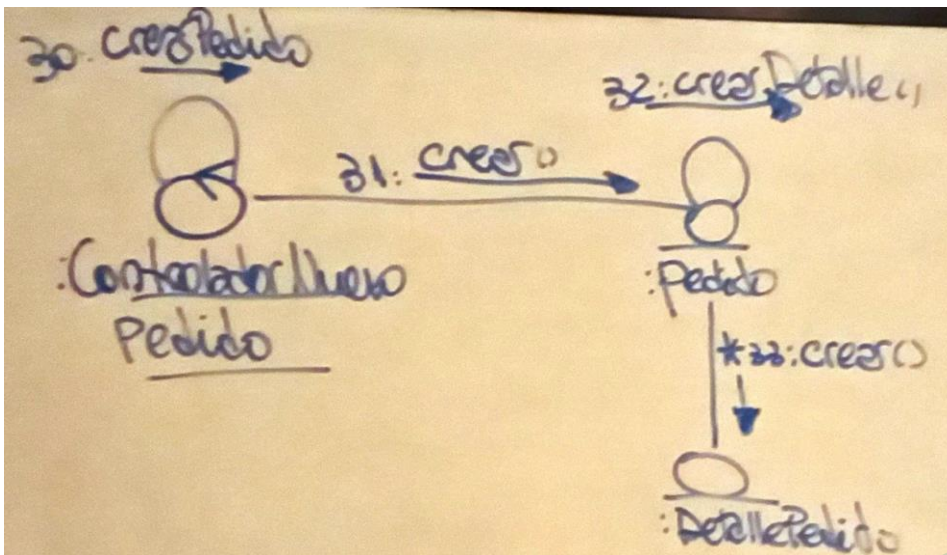
Solución: Asignar a la clase B la responsabilidad de crear una instancia de la clase A en uno de los siguientes casos:

- B agrega los objetos de A.
- B contiene los objetos de A.
- B registra las instancias de los objetos de A.
- B utiliza específicamente los objetos de A.
- B tiene los datos de inicialización que serán transmitidos a A cuando sea creado (así B es un experto respecto de la creación de A).

Así NO:



Así Sí:



Patrón Controlador:

Problema: ¿Quién debería encargarse de atender un evento del sistema/de un CU?

Solución: Un manejador artificial de todos los eventos del sistema de un CU, generalmente denominado "Manejador <Nombre de CU> (controlador de CU)".

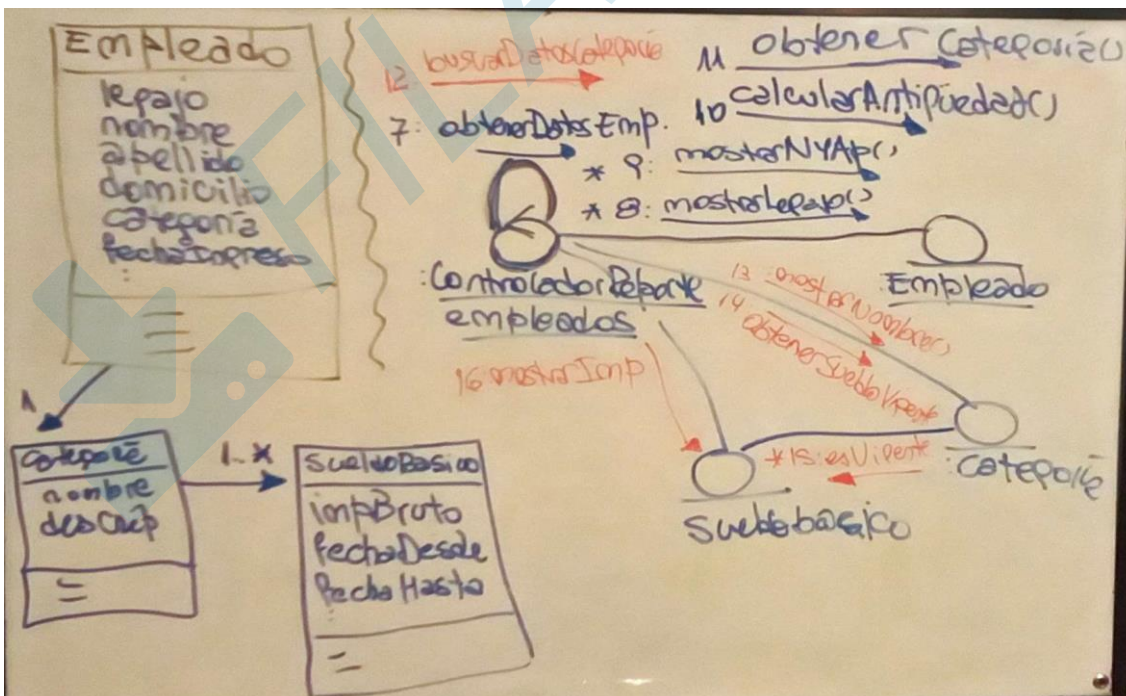
Patrón Bajo Acoplamiento:

Problema: ¿Cómo dar soporte a una dependencia escasa y a un aumento de reutilización?

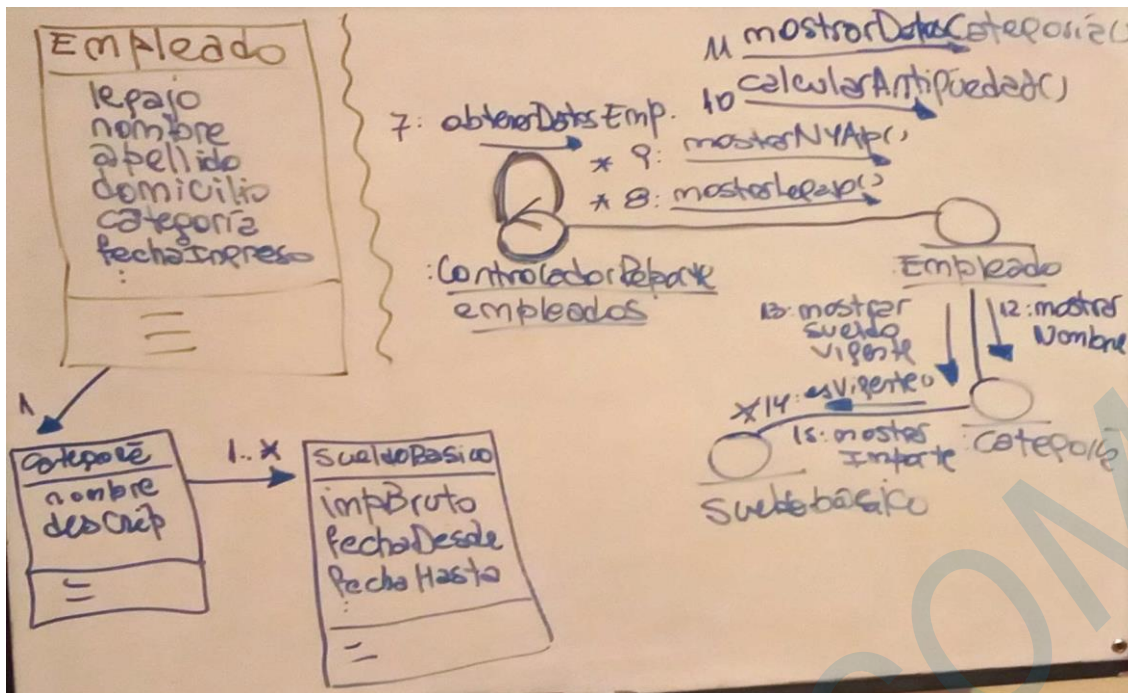
Alto acoplamiento implica: Cambios de las clases afines ocasionan cambios locales, difíciles de entender de manera aislada y de reutilizar porque se requiere la presencia de las clases de las que depende.

Solución: Asignar una responsabilidad para mantener bajo el acoplamiento.

Así NO: (Para método mostrarDatosCategoría())(Genera dependencias entre gestor y categoría y sueldo).



Así Sí:



Patrón Alta Cohesión:

Problema: ¿Cómo mantener la complejidad dentro de límites manejables?

Baja cohesión implica: Son difíciles de entender, reutilizar, conservar y son delicadas (las afectan constantemente los cambios).

Solución: Asignar una responsabilidad de modo que la cohesión siga siendo alta.

Beneficios:

- Se incrementa la claridad y facilita la comprensión del diseño.
- Se simplifican el mantenimiento y las mejoras.
- Se soporta a menudo bajo acoplamiento.
- El grano fino de funcionalidad altamente relacionada incrementa la reutilización porque una clase cohesiva se puede utilizar para un propósito muy específico.